Design a Small Loan Application (MoneyView / KreditBee / Navi)

- Online lending platform that allows users to apply for instant personal loans, complete KYC, check eligibility, and receive loan disbursal.

---

1. Functional Requirement

User Features

- User should be able to register/login into the application.
- User should be able to complete KYC.
- User should be able to check loan eligibility.
- User should be able to apply for a loan.
- User should be able to upload required documents.
- User should be able to repay EMI.
- User should be able to view active loans and repayment schedule.
- User should receive notifications for EMI reminders and loan status.

Admin Features

- Admin should be able to approve/reject loans.
- Fraud detection and risk validation.
- Monitor repayments and defaults.

---

2. Non-Functional Requirement

Scale

- 50M users
- 5M monthly loan applications
- 500K concurrent active users
- 100K TPS peak during campaigns

CAP Theorem

- Availability > Consistency for browsing and eligibility checks.
- Strong Consistency for loan creation, repayment, and ledger updates.

Other NFRs

- High availability (99.99%)
- Idempotent payment APIs
- Secure PII and financial data
- PCI DSS compliance
- Low latency eligibility checks (<300ms)
- Event-driven architecture for async processing
- Audit logging for financial operations

---

3. Identify Core Entities

- User
- LoanApplication
- Loan
- KYC
- CreditScore
- BankAccount
- Repayment
- Transaction
- EMI Schedule
- Notification
- FraudCheck

---

4. API Designing

Authentication APIs

1. POST : /v1/users/register
2. POST : /v1/users/login
3. POST : /v1/users/logout

KYC APIs

1. POST : /v1/kyc/upload
2. GET : /v1/kyc/status/{userId}

Loan APIs

1. POST : /v1/loans/check-eligibility
2. POST : /v1/loans/apply
3. GET : /v1/loans/{loanId}

4. GET : /v1/loans/user/{userId}
5. POST : /v1/loans/{loanId}/cancel

Payment APIs

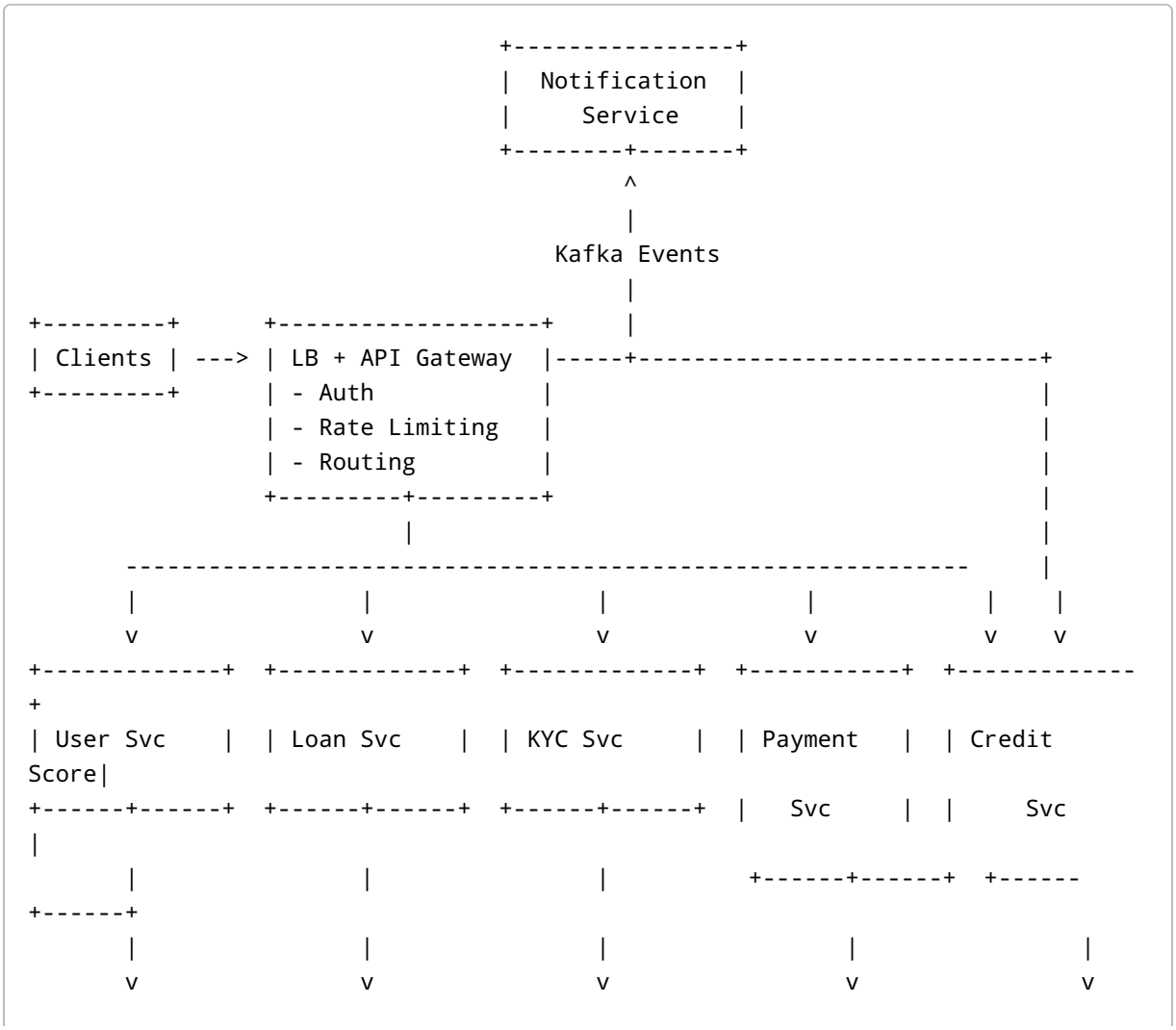
1. POST : /v1/payments/emi
2. GET : /v1/payments/history/{loanId}

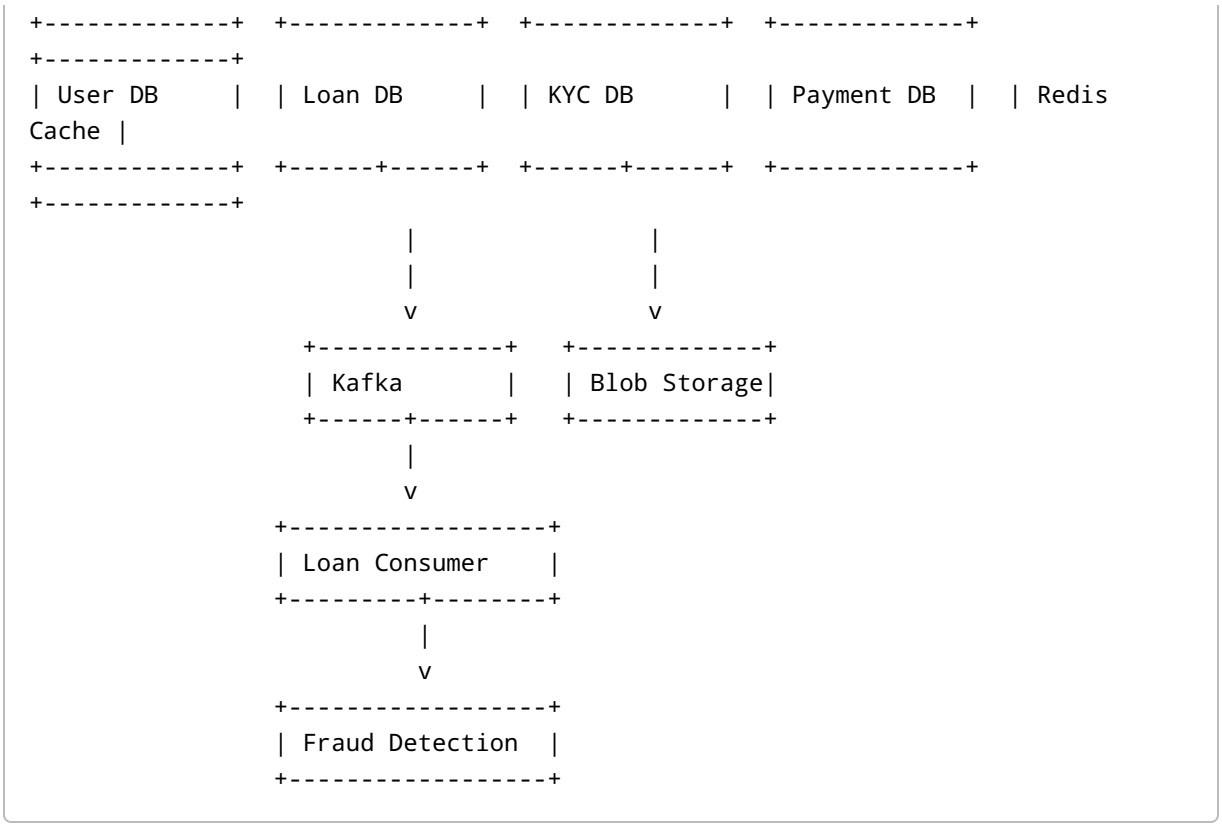
Admin APIs

1. POST : /v1/admin/loan/{loanId}/approve
2. POST : /v1/admin/loan/{loanId}/reject
3. GET : /v1/admin/fraud-checks

---

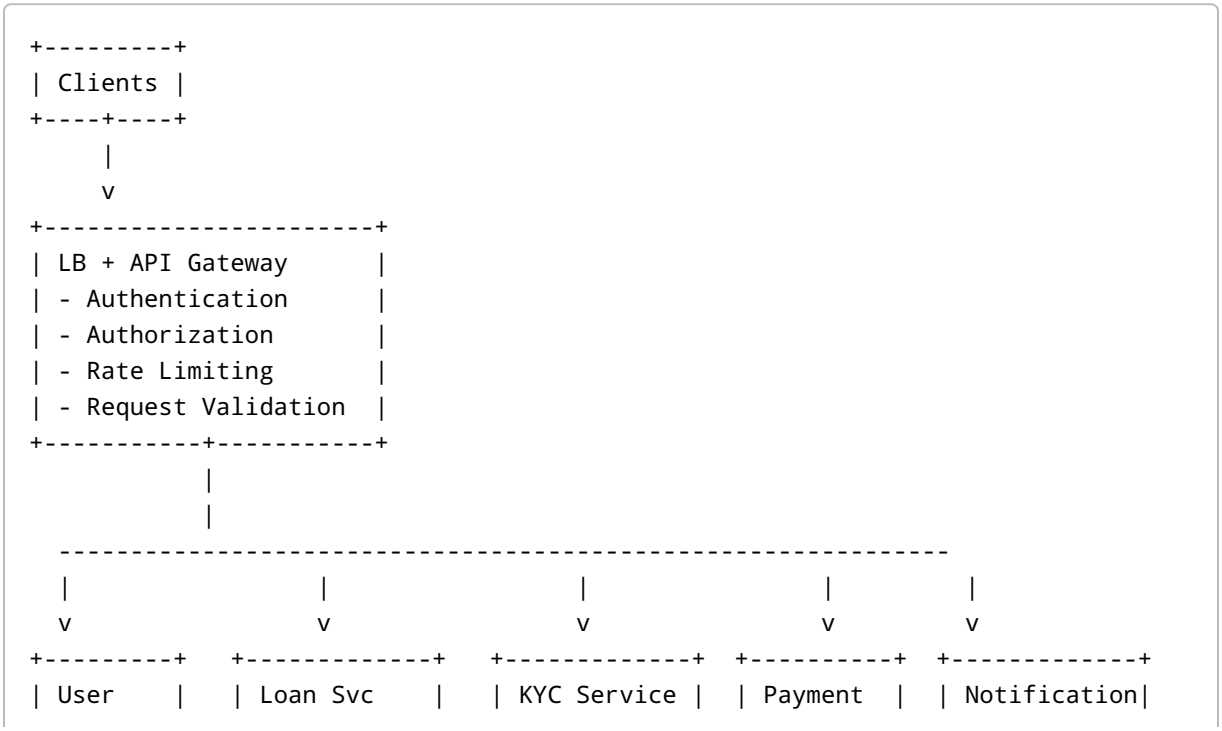
5. High Level Design

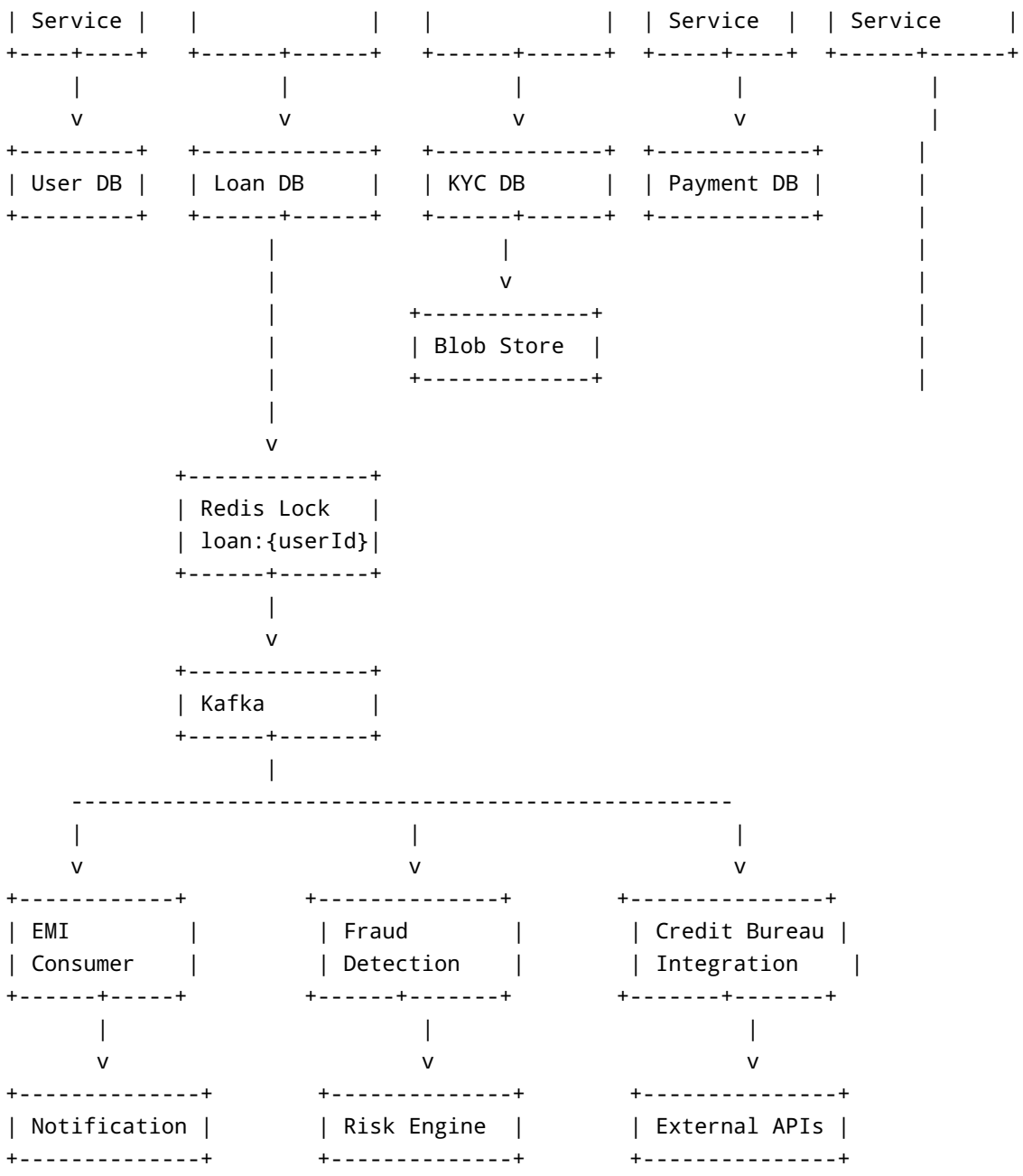




---

6. Low Level Design





---

7. Database Design

User DB (MySQL)

```
User
- userId
- name
```

```
- email
- mobile
- aadhaarNumber
- panNumber
- salary
- createdAt
- updatedAt
```

Loan Application DB (MySQL)

```
LoanApplication
- applicationId
- userId
- amount
- tenure
- interestRate
- status
- createdAt
- updatedAt
```

Statuses:

- PENDING
- APPROVED
- REJECTED
- DISBURSED

Loan DB (MySQL)

```
Loan
- loanId
- userId
- principalAmount
- interestRate
- remainingAmount
- emiAmount
- tenureMonths
- startDate
- endDate
- status
```

Statuses:

- ACTIVE
- CLOSED
- DEFAULTED

EMI Schedule DB

```
EMI_Schedule
- emiId
- loanId
- dueDate
- amount
- status
```

Statuses:

- PAID
- PENDING
- OVERDUE

Payment DB

```
Payment
- paymentId
- loanId
- userId
- amount
- paymentMode
- transactionId
- paymentStatus
- createdAt
```

KYC DB

```
KYC
- kycId
- userId
- aadhaarUrl
```

```
- panUrl
- selfieUrl
- verificationStatus
```

---

8. Detailed Flow — Loan Application

Step 1 — User Checks Eligibility

```
Client -> API Gateway -> Loan Service
                         |
                         +--> Redis Cache
                         |
                         +--> Credit Score Service
                                   |
                                   +--> External Bureau APIs
```

- Credit score fetched from bureau.
- Eligibility rules evaluated.
- Cached in Redis for quick response.

Step 2 — Loan Application

```
Client -> Loan Service
             |
             +--> Redis Distributed Lock
             |
             +--> Loan DB
             |
             +--> Kafka Event
```

Why Redis Lock?

Prevent duplicate applications for same user.

```
lock:user:{userId}
TTL = 5 mins
```

Step 3 — Fraud Detection

```
Kafka -> Fraud Consumer
            |
            +--> Risk Engine
            |
            +--> Device Fingerprinting
            |
            +--> IP Validation
            |
            +--> Blacklist Check
```

If fraud detected:

- Reject loan.
- Notify admin.

Step 4 — Loan Approval

```
Admin -> Loan Service
            |
            +--> Loan DB
            |
            +--> Payment Service
```

Step 5 — Disbursal

```
Payment Service
      |
      +--> Bank Gateway
      |
      +--> Payment DB
      |
      +--> Kafka
```

Notification Service consumes Kafka event and sends:

- SMS
- Email
- Push Notification

9. EMI Payment Flow

```
Client -> Payment Service
             |
             +--> Idempotency Check
             |
             +--> Payment Gateway
             |
             +--> Payment DB
             |
             +--> Loan DB
             |
             +--> Kafka Event
```

10. Rate Limiting

Use Token Bucket Algorithm.

```
Key:
rate_limit:{userId}
```

Why?

- Protect loan APIs from abuse.
- Prevent brute-force OTP attempts.
- Prevent DDoS attacks.

11. Caching Strategy

Redis Cache

Cacheable Data

- Loan eligibility response
- User profile
- Credit score
- EMI schedule
- Loan details

Cache Key Examples

```
user:{userId}
loan:{loanId}
eligibility:{userId}
creditScore:{userId}
```

---

12. Database Choice

Component	DB Choice	Reason
User Data	MySQL	Strong consistency
Loan Ledger	PostgreSQL/MySQL	ACID transactions
Cache	Redis	Low latency
Logs	Elasticsearch	Search and analytics
Documents	Blob Storage (S3)	Cheap scalable storage
Event Queue	Kafka	Async event processing

---

13. Scalability Discussion

Horizontal Scaling

- Stateless microservices.
- Multiple replicas behind load balancer.
- Kafka partitions for parallel processing.
- Redis cluster for distributed caching.

Database Scaling

Read Replicas

- Heavy reporting queries.
- Analytics dashboards.

Sharding

Shard Loan DB by:

```
hash(userId)
```

---

14. Reliability & Fault Tolerance

Retry Mechanism

- Retry external payment APIs.
- Exponential backoff.

Dead Letter Queue

Failed Kafka events pushed into DLQ.

Idempotency

Prevent duplicate EMI payments.

```
idempotency_key
```

---

15. Security

Authentication

- JWT-based authentication.
- OAuth for third-party integrations.

Encryption

- Encrypt Aadhaar/PAN.
- TLS for all APIs.
- Encrypted blob storage.

Audit Logs

Store:

- Loan approvals
- Payment attempts

- KYC actions

---

16. Monitoring

Metrics

- Loan approval success rate
- Payment failure rate
- Fraud detection rate
- Kafka consumer lag
- API latency

Tools

- Prometheus
- Grafana
- ELK Stack

---

17. Deep Dive Questions (Interview)

Q1. How do you prevent duplicate loan applications?

Use Redis distributed lock.

```
lock:user:{userId}
```

Only one application flow allowed at a time.

Q2. How will you handle payment failures?

- Retry mechanism.
- Reconciliation job.
- DLQ for failed events.
- Idempotent payment APIs.

Q3. Why Kafka?

Used for async workflows:

- Notifications
- Fraud checks
- Analytics
- Audit logging

Prevents blocking user APIs.

Q4. Why separate Loan Service and Payment Service?

Because:

- Different scaling requirements.
- Payments require strong consistency.
- Easier fault isolation.
- Independent deployments.

---

18. Interview Flow (How To Drive Discussion)

Step 1

Clarify requirements:

- Loan type?
- Instant or manual approval?
- Scale?
- Fraud checks?

Step 2

Define APIs.

Step 3

Identify entities.

Step 4

Draw high-level architecture.

Step 5

Explain critical flows:

- Loan application
- EMI payment
- Fraud detection

Step 6

Discuss scaling:

- Kafka
- Redis
- DB sharding
- Caching

Step 7

Discuss tradeoffs and CAP theorem.

---

19. Bonus Advanced Topics

Event Sourcing

Store immutable ledger events.

CQRS

Separate read/write models for reporting.

Saga Pattern

Used for distributed transaction handling:

```
Loan Approval -> Payment -> Notification
```

Rollback if disbursal fails.

---

20. Final Architecture Summary

```
Client
  -> API Gateway
       -> User Service
       -> Loan Service
       -> KYC Service
       -> Payment Service
       -> Notification Service

Redis:
- Cache
- Distributed Lock
- Rate Limiting

Kafka:
- Async processing
- Fraud checks
- Notifications
- Analytics

MySQL/Postgres:
- User DB
- Loan DB
- Payment DB

Blob Storage:
- KYC Documents
```